

Coin Change II (Count Ways)

Complete Documentation

Problem Statement

Given an array of coin denominations and a target amount, find the total number of ways to make the target amount.

You can use each coin unlimited times.

Example

Coins = [1, 2, 3]

Amount = 4

Valid combinations:

- 1 + 1 + 1 + 1
- 1 + 1 + 2
- 2 + 2
- 1 + 3

Answer: 4 ways

Key Observation

This is a counting problem, not an optimization problem.

We are not asked:

- Minimum coins?
- Maximum value?

We are asked:

- How many ways?

Therefore, whenever two recursive paths are possible:

- Take
- Not Take

their results are added:

```
return take + notTake;
```

because both paths may contain valid solutions.

Recursive State Definition

Define:

$f(ind, T)$

Meaning:

- Number of ways to make amount T using coins from index 0 to ind .

For:

- Coins = [1,2,3]
- $f(2,4)$

means:

- Number of ways to make 4 using coins {1,2,3}.

Recursive Choices

At every coin we have two choices.

1. Not Take Current Coin

Move to previous coin.

```
notTake = f(ind-1, T)
```

2. Take Current Coin

Since a coin can be reused infinitely:

```
take = f(ind, T-arr[ind])
```

Notice:

- index remains same because the current coin can be used again.

Recurrence Relation

```
f(ind,T) =  
    f(ind-1,T)  
    +  
    f(ind,T-arr[ind])
```

Pattern 1 : Base Case = ind == 0

This is the most common DP solution.

Code

```
long solve(int ind, int T, vector<int>& arr)  
{  
    if(ind == 0)  
        return (T % arr[0] == 0);  
  
    long notTake = solve(ind-1,T,arr);  
  
    long take = 0;  
    if(arr[ind] <= T)  
        take = solve(ind,T-arr[ind],arr);  
  
    return take + notTake;  
}
```

Understanding The Base Case

```
if(ind == 0)  
    return (T % arr[0] == 0);
```

When only one coin remains:

- arr[0]

we check:

- Can the remaining amount be formed using only this coin?

Example:

- Coin = 1
- Target = 4
- $4 \% 1 == 0$
- **Return:** 1 (because: 1+1+1+1 forms the amount.)

Example:

- Coin = 2
- Target = 5
- $5 \% 2 \neq 0$
- **Return:** 0 (because: $2 + 2 + ?$ cannot form 5.)

Special Case

Suppose:

- ind = 0
- T = 0

Then:

- $0 \% \text{arr}[0] == 0$

returns:

- 1

Meaning:

- There is exactly one way to make amount 0. Choose nothing.

This is called the empty combination.

| Pattern 2 : Base Case = target == 0

Another common pattern.

Code

```
long solve(int ind, int T, vector<int>& arr)
{
    if(T == 0)
        return 1;

    if(ind < 0)
        return 0;

    long notTake = solve(ind-1,T,arr);

    long take = 0;
    if(arr[ind] <= T)
        take = solve(ind,T-arr[ind],arr);

    return take + notTake;
}
```

Understanding The Base Cases

Success State

```
if(T == 0)
    return 1;
```

Meaning:

- Target achieved.
- We found one valid combination.

Return:

- 1 valid way

Example

Coins = [1,2]

Take 2

Target becomes:

- $2 - 2 = 0$

We have found:

- [2] A valid combination.

Return:

- 1

Failure State

```
if(ind < 0)
    return 0;
```

Meaning:

- No coins left.
- But target is still positive.

Therefore:

- No valid solution exists.

Return:

- 0

Why Does Amount 0 Return 1?

This is one of the most important DP concepts.

Question: How many ways can you make amount 0?

Answer: Exactly one way.

That way is: Choose nothing.

Representation: {}

Therefore: return 1;

Complete Recursion Tree

Input:

Coins = [1,2,3]

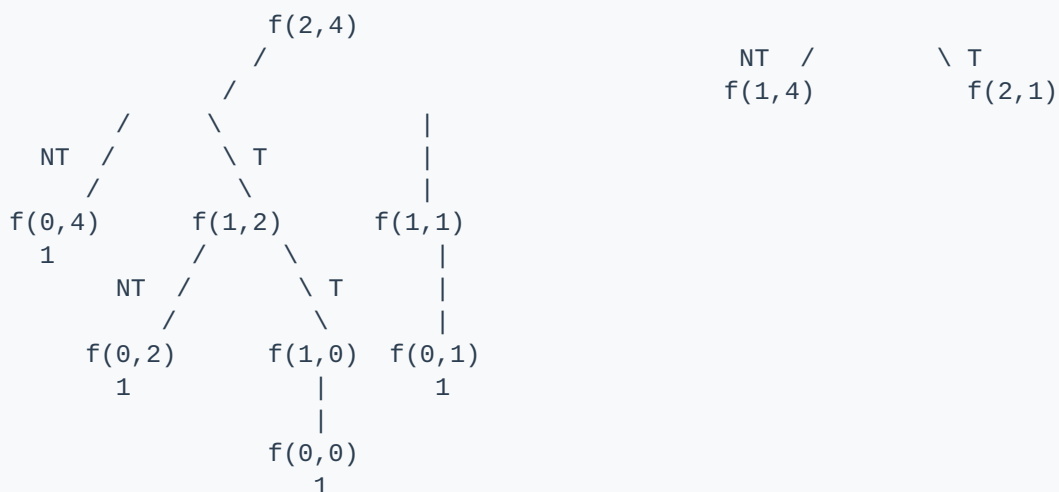
Amount = 4

Initial Call: f(2,4)

Recursion Tree

```
f(2,4)
├── notTake → f(1,4)
│   ├── notTake → f(0,4)
│   │   └── = 1
│   └── take → f(1,2)
│       ├── notTake → f(0,2)
│       │   └── = 1
│       └── take → f(1,0)
│           └── f(0,0)
│               └── = 1
│
│   return 2
return 3
└── take → f(2,1)
    ├── notTake → f(1,1)
    │   └── notTake → f(0,1)
    │       └── = 1
    │
    │   return 2
    return 3
return 4
```

Visual Tree



How Returns Propagate

Step 1

$$f(1,0) = 1$$

Step 2

$$f(1,2) = f(0,2) + f(1,0) = 1 + 1 = 2$$

Ways: 1+1, 2

Step 3

$$f(1,4) = f(0,4) + f(1,2) = 1 + 2 = 3$$

Ways: 1+1+1+1, 2+1+1, 2+2

Step 4

$$f(2,1) = 1$$

Way: 1

Step 5

$$f(2,4) = f(1,4) + f(2,1) = 3 + 1 = 4$$

Final Answer: 4 ways

Combinations Found

- 1 + 1 + 1 + 1
- 1 + 1 + 2
- 2 + 2
- 1 + 3

Total: 4 ways

DP Memoization Solution

```
long solve(int ind, int T,
           vector<int>& arr,
           vector<vector<long>>& dp)
{
    if(ind == 0)
        return (T % arr[0] == 0);

    if(dp[ind][T] != -1)
        return dp[ind][T];

    long notTake =
        solve(ind-1, T, arr, dp);

    long take = 0;

    if(arr[ind] <= T)
        take =
            solve(ind, T-arr[ind], arr, dp);

    return dp[ind][T]
        = take + notTake;
}
```

Time Complexity: $O(N \times \text{Amount})$

Space Complexity: $O(N \times \text{Amount})$

Pattern Recognition Guide

When To Use `ind == 0` Base Case

Use when thinking: *Only one item remains.*

Examples:

- Coin Change II
- Subset Sum
- Count Subsets
- Partition Problems
- Knapsack Variants

When To Use `target == 0` Base Case

Use when thinking: *The moment the goal is achieved, count the solution.*

Examples:

- Combination Sum
- N-Queens
- Maze Paths
- Generate Parentheses
- Backtracking Problems

Final Takeaway

Coin Change II is fundamentally a:

- Take / Not Take + Counting DP Problem

Core recurrence:

- $\text{ways} = \text{take} + \text{notTake}$

Most important concept:

- Target = 0 means Success

Therefore:

- return 1; because one valid combination has been found.